

Strings + Sliding Window Notes (Interview Guide)

1. Reverse Words in a String

Problem

Given a string, reverse the order of words.

Example:

```
Input  : "  hello  world  "  
Output : "world hello"
```

Conditions:

- remove leading spaces
- remove trailing spaces
- multiple spaces should become single space

Approach 1 — Using trim() + split() + reverse

Intuition

We first clean the string. Then separate words. Then reverse the word array. Then join using single space.

Code

```
class Solution {  
    public String reverseWords(String s) {  
  
        s = s.trim();  
  
        String[] words = s.split("\\s+");  
  
        int low = 0;  
        int high = words.length - 1;  
  
        while(low < high){
```

```
        String temp = words[low];
        words[low] = words[high];
        words[high] = temp;

        low++;
        high--;
    }

    return String.join(" ", words);
}
```

Important Lines

trim()

```
s.trim()
```

Removes:

- leading spaces
- trailing spaces

split("\\s+")

```
String[] words = s.split("\\s+");
```

Meaning:

- \s -> whitespace
 - -> one or more spaces

Handles multiple spaces automatically.

String.join()

```
String.join(" ", words)
```

Joins using exactly one space.

Complexity

Time Complexity

$O(n)$

Space Complexity

$O(n)$

Because `split()` creates extra array.

Interview Explanation

First I remove outer spaces using `trim()`.
Then I split words using regex `"\\s+"` to handle multiple spaces.
Then I reverse the words array using two pointers.
Finally I join using single space.

Important Interview Point

This solution is NOT in-place.

Because:

`split()`

creates extra memory.

Approach 2 — In-place Hard Version

Idea

1. Remove extra spaces
 2. Reverse complete string
 3. Reverse each word
-

Complexity

TC = $O(n)$
SC = $O(1)$

Interview Line

The in-place approach is more optimized in terms of space.

2. Longest Substring Without Repeating Characters

=====

LeetCode 3

Problem

Find length of longest substring containing unique characters only.

Example:

Input : "abcabcbb"
Output : 3

Because:

```
"abc"
```

is longest unique substring.

Most Important Concept

This is a classic:

```
Sliding Window Problem
```

Sliding Window Intuition

Maintain a window:

```
[low .... high]
```

such that:

```
all characters inside window are unique
```

When duplicate comes

```
Shrink window from left side
```

Approach 1 — Brute Force

Idea

Generate all substrings. Check each substring for uniqueness.

Complexity

TC = $O(n^3)$

Very bad.

Approach 2 — Sliding Window + HashSet

Intuition

HashSet stores characters currently inside window.

If duplicate found:

remove characters from left
until duplicate disappears

Code

```
class Solution {
    public int longestUniqueSubstr(String s) {

        HashSet<Character> set = new HashSet<>();

        int low = 0;
        int high = 0;

        int ans = 0;

        while(high < s.length()){

            char c = s.charAt(high);

            while(set.contains(c)){
                set.remove(s.charAt(low));
                low++;
            }

        }
    }
}
```

```
        set.add(c);

        ans = Math.max(ans, high - low + 1);

        high++;
    }

    return ans;
}
```

Window Length Formula

```
length = high - low + 1
```

Reason:

If:

```
low = 2
high = 4
```

then elements are:

```
2,3,4
```

Total:

```
3 elements
```

Dry Run

Input:

```
"abcabcbb"
```

Process:

```
[a]
[ab]
[abc]
Duplicate a found
Remove a
Continue
```

Maximum:

3

Why While Loop?

```
while(set.contains(c))
```

Because there may still be duplicates after one removal.

Need to keep shrinking.

Complexity

Time Complexity

```
O(n)
```

Reason:

Each character:

- enters window once
- leaves window once

Space Complexity

$O(256) \approx O(1)$

(for character set)

Interview Explanation

I maintain a sliding window containing unique characters.
If duplicate appears, I shrink the window from left side
until the duplicate is removed.
Then I update maximum window length.

Best Interview Line

Each character enters and leaves the window at most once,
therefore total complexity is $O(n)$.

=====

3. Optimized Sliding Window using HashMap

=====

Why Optimize?

In HashSet approach:

```
while(set.contains(c))
```

we remove characters one-by-one.

Better Idea

Instead of removing one-by-one:

```
jump left pointer directly
```

using previous index.

Main Intuition

HashMap stores:

```
character -> last index
```

Example:

```
'a' -> 5  
'b' -> 7
```

Code

```
class Solution {  
    public int lengthOfLongestSubstring(String s) {  
  
        HashMap<Character, Integer> map = new HashMap<>();  
  
        int i = 0;  
        int ans = 0;  
  
        for(int j = 0; j < s.length(); j++){  
  
            char c = s.charAt(j);  
  
            if(map.containsKey(c)){  
                i = Math.max(i, map.get(c) + 1);  
            }  
  
            map.put(c, j);  
        }  
    }  
}
```

```
        ans = Math.max(ans, j - i + 1);
    }

    return ans;
}
```

MOST IMPORTANT LINE

```
i = Math.max(i, map.get(c) + 1);
```

Why Math.max() Needed?

Because:

```
we NEVER remove entries from map
```

So map may contain:

```
old indices from previous windows
```

Example

```
"abba"
```

When processing final 'a':

Map contains:

```
a -> 0
```

But current window already starts ahead.

Without Math.max():

```
left pointer may move backward
```

which is wrong.

Therefore

```
Math.max()
```

ensures:

```
left pointer only moves forward  
never backward
```

Golden Interview Line

```
Math.max() prevents the left pointer from moving backward  
because HashMap still contains outdated indices.
```

Complexity

Time Complexity

```
O(n)
```

Space Complexity

```
O(256) ≈ O(1)
```

Set vs Map Approach

Feature	HashSet	HashMap
Easier to understand	Yes	Medium
Easier to explain	Yes	Medium
Removes one-by-one	Yes	No
Direct jump possible	No	Yes
More optimized	Slightly less	Better
Best for beginners	Yes	No

Which Should You Use?

For Learning

Use:

HashSet approach

because intuition becomes strong.

For Advanced Optimization

Use:

HashMap approach

Interview Recommendation

Learn in order:

1. Brute Force
2. HashSet Sliding Window
3. HashMap Optimization

This shows growth of thinking.

=====

IMPORTANT SLIDING WINDOW INTERVIEW NOTES

=====

How to Identify Sliding Window Problems

Usually questions contain:

```
substring  
subarray  
continuous segment  
longest  
smallest  
maximum  
minimum
```

Generic Sliding Window Pattern

```
while(high < n){  
    // include current element  
  
    while(condition breaks){  
        // shrink from left  
        low++;  
    }  
  
    // update answer  
  
    high++;  
}
```

Important Observations

Window Expand

```
high++
```

Window Shrink

```
low++
```

Window Length

```
high - low + 1
```

Common Sliding Window Mistakes

Mistake 1

Updating answer at wrong place.

Mistake 2

Using:

```
high - low
```

instead of:

```
high - low + 1
```

Mistake 3

Moving left pointer backward.

Mistake 4

Using if instead of while.

Sometimes multiple removals needed.

Final Interview Tips

While Explaining

Always say:

```
I maintain a valid window.  
If condition breaks, I shrink the window.
```

Mention Complexity Clearly

```
Each character processed at most twice.  
Therefore TC =  $O(n)$ .
```

Best Communication Style

Explain:

1. Window meaning
 2. Why shrink needed
 3. Why while loop
 4. Complexity reasoning
 5. Edge cases
-

Edge Cases to Mention

Empty String

```
""
```

Single Character

```
"a"
```

All Same Characters

```
"aaaa"
```

All Unique

```
"abcdef"
```

Final Golden Lines

For HashSet Approach

```
Shrink window gradually until it becomes valid again.
```

For HashMap Approach

```
Directly jump left pointer using previous index.
```

Most Important Sliding Window Concept

Window should always remain valid.